OR Spectrum (2014) 36:607–629
DOI 10.1007/s00291-013-0332-9

REGULAR ARTICLE

Preference-based assignment of university students to multiple teaching groups

Henrik Heitmann · Wolfgang Brüggemann

Published online: 13 July 2013
© Springer-Verlag Berlin Heidelberg 2013

Abstract A successful approach to the student-scheduling problem is presented here. This problem arises naturally when courses and classes must be offered in such a large number of multiple teaching groups that some of these are timetabled in parallel, i.e. simultaneously. This is the case, for example, at the School of Business, Economics and Social Sciences of Universität Hamburg, Germany. Here, 3,735 students have to be assigned in a real-life scenario to 48 courses in 300 multiple teaching groups such that no student has a time clash and students' enrolments as well as group capacities are met. Students' satisfaction with regard to individually specified preferences for various aspects of the scheduling can be used as the objective which yields a multi-criteria decision problem. The resulting mixed-integer programme is modelled in GAMS and typical instances can be solved within minutes using the commercial CPLEX solver. Furthermore, the integration of the optimisation approach into the existing registration process using a campus-management software is discussed.

Keywords Students-scheduling problem · University timetabling · Academic timetabling · Mixed-integer programming

1 Introduction

One of the most difficult and always recurring organisational problems at large universities and high schools is the assignment of students to courses that are offered in

H. Heitmann (✉) · W. Brüggemann
Institute for Operations Research, Universität Hamburg, Von-Melle-Park 5,
20146 Hamburg, Germany
e-mail: henrik.heitmann@uni-hamburg.de

W. Brüggemann
e-mail: brueggemann@uni-hamburg.de

multiple teaching groups if some of these are scheduled to take place simultaneously. At the time of the assignment, the timetable is already fixed, each of the multiple groups is assigned to a teacher (not necessarily the same for all groups of the same course) and a room with a certain capacity. Moreover, students are enrolled to courses and had the opportunity to specify their preferences for the different groups of their courses. Then students must be assigned to the groups of the courses for which they are registered such that there are no clashes in the individual timetables and the room or group capacities are not exceeded (van den Broek et al. 2009; Laporte and Desroches 1986). The same problem exists at the School of Business, Economics and Social Sciences of Universität Hamburg. After the course timetable is fixed, for example, for the summer semester 2011, 3,735 students have to be assigned to 48 courses in Business Administration which are taught in 300 multiple teaching groups. Each course in the first two years of this programme is mandatory and offered in multiple groups; the number of multiple groups for these courses varies between 2 and 36.

A few years earlier, in the winter semester 2006/07, a new campus-management software (CampusNet by Datenlotsen 2013, locally called STiNE) was introduced for the administration of the newly established Bachelor and Master programmes. Students enrol for their courses through this software which can subsequently be used to automatically assign the students to the multiple groups of their courses. However, there is at this time no built-in functionality included in the software to check that the resulting timetables are clash free for the students. Hence, this automatic assignment of the students to the multiple teaching groups may (and often will) result in individual clashes. To avoid this problem, the university administration divides all students and all multiple teaching groups into sub-cohorts. Subsequently, the timetabling problem is solved for each sub-cohort. The timetables—each sub-cohort has a fixed timetable without clashes—are worked out manually by the university administration. Thus, the students can only choose from a part of the multiple groups offered and certain rather restrictive and otherwise often nonsensical scheduling requirements have to be met by the teaching staff. Due to the large number of offered groups, this splitting up into sub-cohorts yields an unnecessarily reduced flexibility on all sides—students, teaching and administration staff—and as a consequence a loss of welfare.

In 2008, a feasibility study was conducted at the School of Business, Economics and Social Sciences in the Universität Hamburg to automate the course timetabling approach according to Schimmelpfeng and Helber (2007). They describe a course timetabling problem at the School of Economics and Management at Leibniz Universität Hannover which is solved with a mixed-integer programme. The result of the study is that with small adaptations to the authors' approach, it could be used as well for the programmes in Hamburg. However, the approach according to Schimmelpfeng and Helber (2007) cannot be fully utilised in Hamburg as long as the division into sub-cohorts prevails to guarantee feasibility of the subsequent student assignment. Or conversely, there needs to be a reliable solution approach to this student scheduling if a full-fledged automated timetabling (i.e. without sub-cohort restrictions) shall be used in future.

From the students' perspective, the overall satisfaction could be increased if the set of teaching groups from which they are allowed to choose (or at least specify preferences for) would be extended to the whole set of multiple groups offered rather than

only the part designated for the respective sub-cohort. A survey among the students (carried out early in 2009) reinforces this opinion as well. In addition, students stated that it would be desirable to take further structural preferences into account for the assignments. Accordingly, the idea was born to solve the genuinely existing student-scheduling problem, such that the division into sub-cohorts would become obsolete. Therefore, a mixed-integer programme is developed with which the students' assignment to multiple teaching groups can be automated and the feasibility problems with the existing campus-management software can be overcome.

This article is a report about the state of student scheduling at the School of Business, Economics and Social Science of Universität Hamburg. It describes the existing student-scheduling problem and gives a mixed-integer model formulation with the main restrictions for the assignment. In addition, a new approach is presented to make the assignment and the resulting individual timetable more comfortable for the students who can individually specify their preferences for the classes and groups. Results for the real-world scenario of the summer 2011 are complemented by the analysis of experiments solving problem instances with artificially generated preference data.

In Sect. 2 university scheduling and timetabling problems in the literature are briefly reviewed. A mixed-integer formulation of the student assignment problem is described in Sect. 3. Afterwards, it is discussed, when the optimisation should be carried out during the real-life registration process, and the operational optimisation process is introduced and explained. In Sect. 5, computational tests are presented and, finally, conclusions and an overview of further research are given in Sect. 6.

2 Literature review

Several articles deal with university (or school) timetabling and scheduling problems. Daskalaki and Birbas (2005) note that timetabling problems have been studied since the 1960s. Typically university timetabling and scheduling concern the assignment of teachers, lecturers or students to courses, groups, sessions or exams to a limited number of time slots to satisfy given constraints (Lewis and Paechter 2007). Constraints used in timetabling can be categorised into hard and soft constraints (Burke et al. 1997). Hard constraints must not be broken. If such a hard restriction cannot be fulfilled, no feasible solution can be found and the instance of the corresponding timetabling problem is infeasible. Soft constraints may be violated at the cost of a penalty which deteriorates the value of the objective function but keeps the corresponding instance of the timetabling problem itself feasible. Brief overviews follow on the class-teacher model, the university-timetabling problem (especially course and examination timetabling), and finally the student-scheduling problem, respectively.

The class-teacher model is used to assign teachers to all classes of a school, such that each teacher is only assigned to one class and each class is taught by at least one teacher at all times (Daskalaki and Birbas 2005; de Werra 1985; Drexl and Salewski 1997); classes are here defined as disjoint sets.

According to van den Broek et al. (2009), timetabling problems at universities can be divided into two categories. The first category is the examination timetabling (Burke et al. 1997; Carter and Laporte 1996; Schaerf 1999). The goal is to find a

timetable for examinations at universities, such that examinations do not coincide in times and rooms with each other and students have no more than a given number of examinations per day (Lewis and Paechter 2007). In contrast, the course timetabling deals with the scheduling of courses to time slots, during which a course should be held, and rooms, where they are taught (Lewis and Paechter 2007; Schaerf 1999; Schimmelpfeng and Helber 2007). It is particulary important for the timetabling that the resulting schedules contain as few time clashes for the students as possible. However, the numbers of student enrolments to the courses are often not known in advance, i.e. prior to the scheduling. The main difference to the class-teacher model is that it cannot be assumed here that students can be scheduled in disjoint sets as classes. The fixed timetable can then be used as an input or starting point for the next stage of student assignment or scheduling.

For this student-scheduling problem (van den Broek et al. 2009; Cheng et al. 2003; Feldman and Golumbic 1989), the weekly timetable is fixed. Here, different courses exist which are offered in multiple teaching groups some of which may be scheduled in parallel. Each multiple teaching group is scheduled in at least one time slot and the maximum group size is given, for example, by the room capacity. Students are registered to the courses and can specify their preferences for the different teaching groups of their courses. In most cases, each student has to be assigned to exactly one of these teaching groups per course although it is conceivable in principle that a student may have to be enrolled for more than one group of a course. In either case, each student can only be assigned to one group at each time slot and, additionally, maximum group sizes may not be exceeded. Laporte and Desroches (1986) use soft and hard constraints to get a feasible solution for their problem in a large Canadian Engineering School. In this problem, the students' course selection must be respected, but all other requirements are handled as soft constraints, e.g. room capacities should not be exceeded, all groups of the same course should roughly have the same number of students, time conflicts and infeasible moves (two groups scheduled in two consecutive time periods in two different campuses) should be avoided and time for lunch breaks should be admitted. The algorithm for the students' assignment is organised in three phases. In the first phase, student schedules are calculated, then, the group assignments are balanced, and finally, the solution is adjusted as to not exceed the room capacities. Cheng et al. (2003) look at the student-scheduling problem for a typical North American high school. At first, they employ hard constraints to guarantee conflict-free schedules for all students on an individual basis and prove that the feasibility version of this problem is already NP-complete for a single student. (Technically, the proof of their Theorem 1 is by restriction to—rather than "reduction to"—the independent set problem which yields, as needed, a polynomial transformation and hence a Turing reduction in the opposite direction as stated (Garey and Johnson 1979, p. 53 and 111).) Since this single-student version of the student-scheduling problem is also a special case of the problem examined here in this paper, it follows that its optimisation version is NP-hard. Afterwards, Cheng et al. (2003) observe that their formulation with hard constraints does not work with real world problems because of infeasible student enrolments. Subsequently, they discuss this limitation and present possible methods to resolve these infeasibilities. In a European university context, van den Broek et al.

(2009) suggest an integer-programme formulation for the student-scheduling problem of the Department of Industrial Design at TU Eindhoven. The problem is solved in four stages with different objectives at each stage.

3 Student scheduling as an assignment problem

Student scheduling can be modelled as an assignment problem with additional constraints. The main scheduling requirements are that the individual timetables are feasible, i.e. without clashes, and that students' enrolments to courses are fulfilled as well as group capacities are not exceeded. The constraints resulting from these requirements have to be met and suffice to define a basic student-scheduling problem. This basic problem is introduced first after the underlying assumptions are given in detail. As a consequence of students' responses in the survey mentioned already in Sect. 1, the basic model can be expanded to incorporate students' preferences into the scheduling such as group preferences, time for lunch breaks, maximum number of days present at the university and core time preferences. These preferences and further requirements by the administration staff are included accordingly as contributions to an objective function or soft restrictions in the resulting extended mixed-integer programme which is presented after the basic student-scheduling problem.

3.1 Assumptions

There are three categories of assumptions: the first category covers all aspects regarding the courses, groups and the given schedule as well as time and rooms. In the second category, the assumptions pertaining to the students can be found while the last category contains the staff—teaching and administrative—assumptions. The formal notation and symbols for variables, parameters and data is introduced along the way while a comprehensive list of symbols can be found in the appendix in Table 5 for easy reference.

Courses, groups, schedule, time, and rooms

C1 A course $j \in J$ is taught in one or multiple teaching groups $k \in K_j$ where J is the set of all courses and K_j is the set of all teaching groups of course j. The set of all teaching groups, i.e.

$$K = \bigcup_{j \in J} K_j$$

is the (disjoint) union of the sets of groups over all courses $j \in J$.

C2 All groups for all courses are scheduled a priori, i.e. the schedule is fixed.

C3 Time is discretised into time slots h and days d according to this given schedule.

C4 It is possible for a group to be taught during multiple time slots per week which need not be connected.

C5 This given schedule is encoded as binary data t_{jkhd} where $t_{jkhd} = 1$ if group k of course j is scheduled to be taught in time slot h of day d.

C6 Some of the multiple teaching groups of a course may be scheduled in parallel, i.e. during the same time period. In contrast, some of the multiple teaching groups are taught by the same member of staff and must hence not be scheduled in parallel.
C7 Each teaching group is a priori assigned to a room. The minimum of the size of the assigned room and an also specified maximum group size yields a given group capacity q_{jk} for group k of course j.
C8 The total group capacity for all multiple groups of a course is sufficient to accommodate all students registered for that course.
C9 The schedule allows enough transition time to get from one teaching group to another if these groups are not scheduled to be held in parallel or to overlap.

Students

S1 Each student who is at least registered for one course $j \in J$ is individually known as a student $i \in I$ where I is the set of all such students.
S2 A student i is a priori registered for her or his courses. This registration information is given as the set of student i's courses $J_i \subseteq J$. Conversely, the set of students who are registered for course j is given as $I_j \subseteq I$.
S3 Each student has the opportunity to specify preferences and relative weights regarding
  S3a the multiple groups offered for each class for which the student is registered
  S3b lunch breaks
  S3c maximum number of days at the university per week
  S3d core times
  before the optimisation is started. If no preferences or weights are specified by the student, these quantities are set to default values.

Staff (teaching and administration)

A1 The resulting timetable for each student i must not exhibit any clashes or overlaps.
A2 It is possible for the administration to specify (block out) time periods during which a student is unavailable for her or his courses (e.g. if the student is needed as a tutor during that time slot). If this is the case for student i in time slot h of day d this is indicated by the blocked-time-slot indicator $b_{ihd} = 1$.
A3 The teaching and/or administrative staff may want to balance the number of students assigned to multiple groups of a course in relation to the given group sizes. This is indicated and facilitated if the administration specifies a minimal group-capacity utilisation factor κ_{jk} as a fraction of the given group capacity q_{jk} for group k in course j.
A4 Teaching and administrative staff may want to impose an upper bound on the maximal unhappiness of students with regard to their preference satisfaction.

3.2 The basic problem

For the basic student-scheduling problem, only three sets of hard constraints need to be defined, and if these three sets of restrictions are all satisfied, the student-scheduling

problem is feasible. The basic student-scheduling problem can be reduced to the assignment of students to multiple teaching groups of courses ensuring the students' enrolments are met, the compliance with maximum group capacities and avoiding time clashes on an individual basis (including time-slot blocking). As already introduced in the previous subsection, different indices, (sub-)sets and input data are needed. As a brief summary, the main problem elements are

- students $i \in I$ (see assumption S1),
- courses $j \in J$ with multiple groups $k, \hat{k} \in K_j \subseteq K$ (see assumption C1),
- time slots $h \in H$ and days $d \in D$ (see assumption C3).

The schedule data are encoded by the binary group-time-slot links t_{jkhd} which indicate whether group k of course j is taught in time slot h on day d by $t_{jkhd} = 1$. For each group k of course j, a group capacity q_{jk} is specified as the maximal number of students who can be assigned to the corresponding group. In addition, a time slot h on day d can be individually blocked for student i with blocked-time-slot indicator $b_{ihd} = 1$, such that student i cannot be assigned to groups in the corresponding time slot. A clash can result only if there is more than one group available for student i in time slot h of day d. This can be deduced from the registration and schedule information and is indicated by the binary clash indicator $c_{ihd} = 1$. If, conversely, the clash indicator $c_{ihd} = 0$, then student i can only be assigned to at most one group in the specified time slot h on day d.

The main binary decision variable on which the student scheduling is based, is the assignment variable

$$x_{ijk} = \begin{cases} 1 & \text{if student } i \text{ is assigned to group } k \text{ of course } j, \\ 0 & \text{otherwise.} \end{cases}$$

Then the three sets of constraints

$$\sum_{k \in K_j} x_{ijk} = 1 \qquad \forall i \in I,\ j \in J_i \quad (1)$$

$$\sum_{j \in J_i} \sum_{k \in K_j} t_{jkhd} \cdot x_{ijk} \leq 1 - b_{ihd} \qquad \forall i \in I,\ h \in H,\ d \in D \,|\, c_{ihd} = 1 \vee b_{ihd} = 1 \quad (2)$$

$$\sum_{i \in I_j} x_{ijk} \leq q_{jk} \qquad \forall j \in J,\ k \in K_j \quad (3)$$

form the basic student-scheduling problem. Constraints (1) make sure that each student i is assigned to exactly one group k of each course j for which this student is registered, see assumption S2. According to assumption A1, restrictions (2) guarantee that every student i can be scheduled to at most one group k at each time slot h on each day d. If it is indicated that the corresponding time slot is blocked by $b_{ihd} = 1$, the right-hand side of this restriction is equal to zero such that student i cannot be assigned to any group k in the corresponding time slot h of day d—as required in

assumption A2. These restrictions guarantee clash-free assignments on an individual basis for each student i and need only be defined for i–h–d–combinations for which the clash-indicator is evaluated to $c_{ihd} = 1$ or if the blocked-time-slot indicator is set to $b_{ihd} = 1$ for the corresponding student-time-slot-day combination. Finally, the maximum group size of each group k in every course j cannot be exceeded by restrictions (3) which keep the total number of students i assigned to this group below the group capacity q_{jk} as required in assumption C7.

This basic student-scheduling problem consisting of these three sets of hard constraints can readily be used to assign students to teaching groups clash free on an individual basis. No objective function is needed for this basic version of student-scheduling yet. The question whether a feasible solution exists for this basic version is a natural decision problem and—given a certificate in the form of a feasible value assignment for all x_{ijk} variables—a yes-instance can obviously be verified in polynomial time by evaluating the three sets of constraints such that this problem belongs to the class NP. Moreover, the polynomial transformation employed by Cheng et al. (2003) can be used identically here as well. To show that their student-scheduling problem is NP-hard they map an instance of the independent set problem to a special case of their problem which is at the same time also a special case of the basic student-scheduling problem analysed here. It can be concluded that the feasibility version of the basic student-scheduling problem is already NP-complete.

3.3 Extensions

In this subsection, extensions are presented for the basic student-scheduling problem from the previous subsection 3.2. Although this basic version of the problem can already be used to assign students to teaching groups, it is easily conceivable that acceptance for such an automated assignment can greatly be improved if students' preferences are taken into account as well. For example, an individual group preference π_{ijk} of student i taking course j for parallel group k could be expressed as a cost to be minimised. From this student's perspective, it is then desirable to minimize the resulting total group preference

$$\Pi_i = \sum_{j \in J_i} \sum_{k \in K_j} \pi_{ijk} \cdot x_{ijk} \qquad \forall i \in I \quad (4)$$

actually realised by the assignment. Collectively, this yields an objective function

$$\min \sum_{i \in I} \Pi_i \qquad (5)$$

which minimises the total cost of actual assignments for all students.

Building on the basic problem along with this first preference-optimising objective function, a first extension is introduced to include requirements from the administration's side. Afterwards, extensions pertaining to further students' preferences are formulated.

From an administrative point of view, it may be sensible to balance the number of students assigned to each of the multiple groups of the same course—see assumption A3. Of course, this balance of group size or capacity utilisation will depend on the various maximum group sizes q_{jk}. Soft constraints

$$\sum_{i \in I_j} x_{ijk} + s_{jk} \geq \kappa_{jk} \cdot \frac{q_{jk}}{\sum_{\hat{k} \in K_j} q_{j\hat{k}}} \cdot |I_j| \qquad \forall j \in J,\ k \in K_j | \kappa_{jk} > 0 \quad (6)$$

aim for a minimum number of students to be assigned to group k of course j which is specified by the product of the minimal group-capacity-utilisation factor κ_{jk}, the number of all students registered for the class ($|I_j|$), and the relative group capacity which is given as the fraction of the maximum capacity of group k divided by the total capacity available for the whole course ($q_{jk} / \sum_{k \in K_j} q_{jk}$). If this minimum group utilisation of group k in course j cannot be met, it shall still be possible to keep the problem feasible such that a compensation variable $s_{jk} \geq 0$ is needed which links such a violation with a penalty in the objective. The restrictions (6) need only to be formulated if the minimal group-capacity utilisation factor is specified $\kappa_{jk} > 0$.

In addition, the new approach can be used to make the schedule resulting from the assignment more comfortable for students if they specify their individual preferences. This specification is facilitated by the opportunity for student i to return positive values for the weights λ_i^* for the different objective contributions instead of their default value of zero where the asterisk is replaced in the following by the corresponding symbol of the preference considered. For example, a set of lunch-break time slots $L_{id} \subseteq H$ may be specified together with weight λ_i^l for student i on day d. Then soft constraints

$$\sum_{j \in J_i} \sum_{k \in K_j} \sum_{h \in L_{id}} t_{jkhd} \cdot x_{ijk} - l_{id} \leq |L_{id}| - 1 \qquad \forall d \in D,\ i \in I | \lambda_i^l > 0 \quad (7)$$

gather the information whether there is time for a lunch break during this set of lunch-break time slots L_{id} for student i on day d. If in each time slot during the time frame for a lunch break a teaching group is scheduled, i.e. there is no time for a lunch break, the lunch-break-violation variable $l_{id} \geq 0$ is forced to a positive value which is again connected with a penalty term in the objective.

Moreover, students may want to express a preferred maximal number of days m_i at the university. To accommodate such preferences, additional binary variables

$$y_{id} = \begin{cases} 1 & \text{if student } i \text{ is assigned to at least a teaching group on day } d, \\ 0 & \text{otherwise,} \end{cases}$$

are needed as well as the corresponding weight λ_i^r. These presence variables y_{id} are used in constraints

$$\sum_{j \in J_i} \sum_{k \in K_j} \sum_{h \in H} t_{jkhd} \cdot x_{ijk} \leq |H| \cdot y_{id} \qquad \forall d \in D,\ i \in I | \lambda_i^r > 0 \quad (8)$$

$$\sum_{d \in D} y_{id} - r_i \leq m_i \qquad \forall i \in I | \lambda_i^r > 0 \quad (9)$$

to calculate the days-per-week-violation variable r_i if the number of actual presence days at the university is more than the preferred number of days m_i.

Finally, a student may want to be assigned to groups on day d either during morning core times or during afternoon core times. It should be noted here that it is nonsensical to demand assignments during both core times simultaneously. Hence, students can only specify at most one of these options at a time. Morning core time for student i on day d is between the first time slot and morning-core-time end e_{id}^M, while the afternoon core time begins at the afternoon-core-time start f_{id}^A and ends with the last time slot. (Of course, all students could be assumed to have the same morning and afternoon core time windows on every day of the week. But allowing student- and day-specific values here gives a greater flexibility to accommodate individual preferences.) For each student i who specifies such a preference by returning a positive value for either one of the corresponding weights $\lambda_i^{w^M} > 0$ or $\lambda_i^{w^A} > 0$, this yields a set of morning core times $H_{id}^M = \{h \in H | h \leq e_{id}^M\} \subseteq H$ or a set of afternoon core times $H_{id}^A = \{h \in H | f_{id}^A \leq h\} \subseteq H$ for each day d of the week. An assignment outside these preferred core times should not render the scheduling infeasible; such an assignment should instead incur another penalty in the objective function and is flagged up for student i by the morning-core-time-violation variables $w_i^M \geq 0$ or the afternoon-core-time-violation variables $w_i^A \geq 0$, respectively. The later (or earlier) the assignment of such a student i on day d to a teaching group is outside the preferred core time, the larger is the value of the corresponding violation variable as such a violation is multiplied in restrictions

$$\sum_{j \in J_i} \sum_{k \in K_j} \sum_{d \in D} \sum_{h \in H \setminus H_{id}^M} (h - e_{id}^M) \cdot t_{jkhd} \cdot x_{ijk} = w_i^M \qquad \forall i \in I | \lambda_i^{w^M} > 0 \quad (10)$$

or

$$\sum_{j \in J_i} \sum_{k \in K_j} \sum_{d \in D} \sum_{h \in H \setminus H_{id}^A} (f_{id}^A - h) \cdot t_{jkhd} \cdot x_{ijk} = w_i^A \qquad \forall i \in I | \lambda_i^{w^A} > 0 \quad (11)$$

by the corresponding distance in time which is given by $(h - e_{id}^M)$ or $(f_{id}^A - h)$, respectively.

Summarising, the assignment should be made with the intention to satisfy the students on an individual basis as much as possible. Initially, the objective function (5) is formulated to optimise the total preferences of actually realised assignments. The penalty terms for the violations of student i's additional preferences which may consist of

- l_{id} for lunch break violations on day d,
- r_i for the excess of the maximum number of days at the university and
- w_i^M or w_i^A for core time violations

Table 1 Weights and input for normalisation

Symbol	Description
λ_i^{Π}	Weight for normalised magnitude of group preferences for student i
λ_i^{l}	Weight for normalised magnitude of violations of lunch-break constraints for student i
λ_i^{r}	Weight for normalised magnitude of violation of maximum-number-of-days-per-week-at-the-university constraint for student i
$\lambda_i^{w^M}$	Weight for normalised magnitude of violations of morning-core-times constraints for student i
$\lambda_i^{w^A}$	Weight for normalised magnitude of violations of afternoon-core-times constraints for student i
$\underline{\Pi}_i$, $\overline{\Pi}_i$	Best and worst magnitude of group preferences for student i
$\underline{l}_i$, $\overline{l}_i$	Best and worst magnitude of violations of lunch-break constraints for student i
$\underline{r}_i$, $\overline{r}_i$	Best and worst magnitude of violation of maximum-number-of-days-at-university constraint for student i
$\underline{w}_i^M$, $\overline{w}_i^M$	Best and worst magnitude of violations of morning-core-times constraints for student i
$\underline{w}_i^A$, $\overline{w}_i^A$	Best and worst magnitude of violations of afternoon-core-times constraints for student i

must be suitably incorporated into this objective function. To make these different contributions to the objective comparable, each violation is normalised to the interval [0; 1], such that different goal violations can be compared with each other. This is accomplished by estimating lower and upper bound values for the individual violations; for example, there cannot be fewer than zero lunch-break violations while the maximum total violation cannot exceed the number of days per week. The estimates for the remaining lower and upper bounds are derived similarly. The deviation from the best possible value is then set in relation to the difference between the values for the worst and the best case. The notation for the input needed for this normalisation is given in Table 1 along with the already introduced weights for the subsequent weighting.

The assignment is then based on a multi-criteria decision for each student which includes the various individual preferences (assumption S3). Each such normalised total preference or preference violation is multiplied by the corresponding weight λ_i^{Π}, λ_i^{l}, λ_i^{r}, $\lambda_i^{w^M}$ or $\lambda_i^{w^A}$. The total of these weighted preferences or preference violations (French 1989, p. 128) then yields the individual objective value contribution

$$z_i = \lambda_i^{\Pi} \cdot \frac{\Pi_i - \underline{\Pi}_i}{\overline{\Pi}_i - \underline{\Pi}_i} + \lambda_i^{l} \cdot \frac{(\sum_{d \in D} l_{id}) - \underline{l}_i}{\overline{l}_i - \underline{l}_i} + \lambda_i^{r} \cdot \frac{r_i - \underline{r}_i}{\overline{r}_i - \underline{r}_i} + \lambda_i^{w^M} \cdot \frac{w_i^M - \underline{w}_i^M}{\overline{w}_i^M - \underline{w}_i^M} + \lambda_i^{w^A} \cdot \frac{w_i^A - \underline{w}_i^A}{\overline{w}_i^A - \underline{w}_i^A} \qquad \forall i \in I \quad (12)$$

for student i. The newly expanded objective then collects all individual objective contributions to yield the total (students') welfare.

At this point, there are two remarks noteworthy: firstly, if all of a student's preferences can be met, the best resulting individual objective contribution z_i is equal to zero since all preferences are now modelled as normalised costs. Secondly, on the level of an individual student, the variables Π_i given in (4) only need to be defined and the restrictions out of the set of constraints (7)–(11) only need to be formulated and evaluated if the corresponding weights are specified by the student to differ from a value of zero. This can considerably narrow down the problem size.

While it is technically possible to optimise global welfare in this way by just looking at the sum of the individual welfare contributions this utilitarian principle may not be ethically sound (e.g. Young 1995, pp. 9–10): the individual objective contributions of just a few students might receive great harm to improve the overall total welfare. To prevent this, the deviation of each contribution z_i from the individually optimal solution for student i is restricted by an externally given individual upper bound $\overline{z}_i$, such that the optimisation of global (all students') welfare cannot be carried out at too high costs for only a few students, as stated in assumption A4. This is reflected in further soft restrictions

$$z_i - u_i \leq \overline{z}_i \qquad \forall i \in I \tag{13}$$

which may again be broken by forcing variable u_i to a positive value. Such a violation would add a further penalty to the overall objective. As a consequence, this overall objective function can now be completed to

$$\min \sum_{i \in I} z_i + \sum_{i \in I} \alpha_i \cdot u_i + \sum_{j \in J} \sum_{k \in K_j} \beta_j \cdot s_{jk} \tag{14}$$

which balances students' and administrative requirements. Accordingly, the objective function consists of the sum of the individual contributions z_i for all students i, of the total individually α_i-weighted upper-bound-violation compensation u_i, and of the sum of the β_j-weighted penalty for compensations of underfull group capacity utilisations s_{jk}. It is noteworthy that all variables used in the objective function can be directly derived from the original decision variables x_{ijk} which determine the assignments.

4 Implementation

In this section, the implementation and, in particular, the integration into the registration process of a semester are described as well as the operational optimisation process for the Business programmes at Universität Hamburg.

4.1 Integration into the timing of a semester

The teaching at Universität Hamburg is organised in semesters. Typically, the teaching term of a semester begins between a first and a second registration phase and there is a 1-week time window between these two phases. Before the first phase begins, administration has modelled the fixed course timetable in the campus-management

Fig. 1 Operational optimisation process stages

software. During the first phase, the students register for their courses and specify their preferences for the multiple groups using the campus-management software. Moreover, students may specify their individual preferences (weights for the individual goals). A total of 100 points can be allocated here to the different goals which should affect the calculation of their individual timetables. These allocated points are used as individual weights λ_i^*. As a default, all points are allocated to λ_i^Π, such that individual timetables are calculated on the basis of parallel group preferences π_{ijk} only, if no other preferences are specified (assumption S3). Before the teaching term begins, students are scheduled by the optimisation. Finally, the second registration phase ("drop and add") follows, in which remaining group capacities can be assigned manually, while missed enrolments may be made up and incorrect enrolments can be corrected on a first-come first-served basis.

4.2 Operational optimisation process

The optimisation is laid out to be used in combination with the campus-management software. This process is organised in four stages. These consecutive process stages—data export, feasibility test, optimisation and data import—are depicted in Fig. 1. Each stage can only be started if the previous stage is finished.

The first process stage is data export from the campus management software. This export includes the courses offered in the current semester, corresponding multiple teaching groups (along with their capacities and time slots), students, their enrolments and group preferences. Moreover, time-slot blocks need to be specified for student tutors and interdisciplinary students who can only be assigned within allowed time windows.

The second process stage is the feasibility test. The goal of this stage is to detect infeasible student enrolments and to adjust them, such that the problem is feasible in the next stage. For this purpose, feasibility is tested by solving the optimisation problem

$$\min \sum_{i \in I} \sum_{j \in J_i} p_{ij} \tag{15}$$

s.t. restrictions (2), (3) and

$$\sum_{k \in K_j} x_{ijk} + p_{ij} = 1 \qquad \forall i \in I,\ j \in J_i \tag{16}$$

$$x_{ijk} \in \{0, 1\} \qquad \forall i \in I,\ j \in J_i,\ k \in K_j \tag{17}$$

$$p_{ij} \geq 0 \qquad \forall i \in I,\ j \in J_i. \tag{18}$$

In contrast to the sets of restrictions (2) and (3) which identically stem from the basic version of the students-scheduling problem, restrictions (1) are reformulated here to soft constraints (16) with auxiliary variables p_{ij}. This variable will be activated if student i cannot be assigned to any group of course j for which this student is registered. The auxiliary objective (15) can then be used to minimise the number of such infeasible enrolments.

As a result, infeasible enrolments can be reported to the administration, such that they can be examined and resolved with the affected students. If this is not possible in some cases, the variables p_{ij} can be used to adjust the student's enrolments for the optimisation in the third process stage.

This third stage in the operational optimisation process is the main students scheduling. Any parameters remaining to be set are specified. Afterwards, the assignments according to the enrolments are calculated using the student-scheduling model from Sect. 3.

Results of the operational optimisation process are the detection of the infeasible student enrolments and the preference-based assignment of the students to one of the multiple teaching groups of the courses for which they are registered.

5 Computational tests

The computational tests for the basic model as well as for the model extensions are carried out with the same real-life timetabling and registration data. This underlying common framework is laid out first, before the question is addressed whether the basic model can be solved in operable time. Afterwards, the extended model is analysed for which preference data need to be generated artificially as these are not yet available from the real-life system.

5.1 Common framework for basic-model tests and experiments with model extensions

The timetabling and registration data common to all experiments are compiled from the Bachelor programme in Business Administration of the School of Business, Economics and Social Sciences at Universität Hamburg for the summer semester 2011 (Universität Hamburg, Administration 2011). There are

- 3,735 students,
- 48 courses,
- 300 parallel teaching groups ($\sum_{j \in J} |K_j|$),
- 22,597 enrolments ($\sum_{j \in J} |I_j|$) of students to courses,
- 11 discrete one-hour time slots (specified by their beginning) and
- five teaching days

in the common framework of these experiments. The number of enrolments to courses per student ranges between 1 and 23 with a median of just four registrations; its frequency distribution is given in Fig. 2.

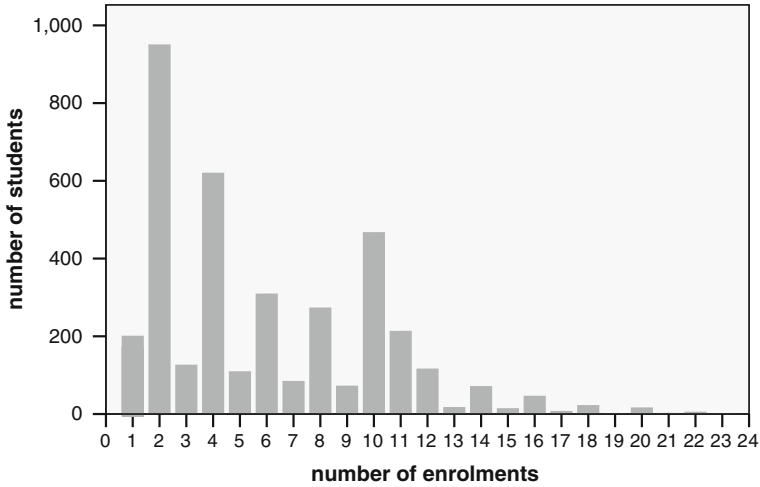

Fig. 2 Frequency distribution of student enrolment to courses

The number of teaching groups per course varies between 1 and 36, and there are between 0 and 18 teaching groups in the same time slot. The majority of these multiple groups is taught in only one or two hours per week such that the chosen discretisation of one-hour time slots appears to be a sensible choice.

The input data then consists of the set of students I, the set of courses J and, for each course $j \in J$, the set of groups K_j along with information about the corresponding teaching times and group capacities as well as the students' enrolments to the courses. From the given list of enrolments, the set of student i's courses J_i can easily be specified, and vice versa, the set of students who are registered for course j is then given by I_j. The set of hours H and the set of days D result from the teaching times of the groups in K.

All tests are carried out in GAMS 22.9.2 and solved with CPLEX 11.2.0 with two parallel threads in Windows 7 Enterprise on a 3.0-GHz Intel Core2 Duo machine with 4-GB RAM.

5.2 Basic model with real-life data

The purpose of the basic problem is to find a feasible solution for the students scheduling. The problem sizes of the mixed-integer programmes for this instance are listed in Table 2 for the feasibility test [objective (15) and restrictions (2), (3) and (16)–(18)] and the basic students-scheduling problem [restrictions (1)–(3)] before and after the CPLEX presolve. Despite of these rather large problem sizes, an optimal integer solution is calculated by the CPLEX solver in 24.46 s for the feasibility-test instance (identifying 152 infeasibilities) and, for the students-scheduling instance, an integer solution is found in 32.05 s. These encouraging results indicate that the basic student-scheduling problem can readily be solved if sub-cohorts are abandoned, but do not yet include any preference data.

Table 2 Problem sizes before and after CPLEX presolve of the feasibility test and the basic students-scheduling mixed-integer programmes, respectively

	Feasibility test		Basic students scheduling	
	Before presolve	After presolve	Before presolve	After presolve
Rows	117,512	76,594	117,353	67,838
Columns	256,832	253,731	233,977	224,655
Non-zeros	841,950	718,987	970,897	662,865
Discrete columns	234,234	233,154	233,976	224,575

Table 3 Probability distribution for the artificial generation of group preferences π_{ijk} depending on time slot

π	8	9	10	11	12	13	14	15	16	17	18	19
0	0 %	0 %	10 %	10 %	20 %	20 %	20 %	20 %	10 %	10 %	0 %	0 %
1	30 %	30 %	30 %	30 %	30 %	30 %	30 %	30 %	30 %	30 %	30 %	30 %
2	30 %	40 %	30 %	40 %	30 %	40 %	40 %	30 %	40 %	30 %	40 %	30 %
3	40 %	30 %	30 %	20 %	20 %	10 %	10 %	20 %	20 %	30 %	30 %	40 %

5.3 Extended model with artificially generated preference data

Among many possible and interesting aspects, three questions are identified which shall be addressed in particular by the tests of the model extensions. The overall layout of these tests is described first along with the additional input required and the default settings for the parameters. Afterwards, the test results are discussed.

5.3.1 Test design, additional input, and default parameter setting

Beyond the deterministic timetabling and registration data from Sect. 5.1, the objective function (14) of the extended model requires preference data which are not yet available. Hence, group preferences and, where necessary, different weights λ_i^* need to be generated artificially. Because of the same contents of teaching, the multiple groups mainly differ from the students' perspective in start times (and possibly instructors). Hence, the probabilities for the artificial generation of group preferences should mainly depend on the time slot. Assuming students would rather be assigned to courses which start in the middle of the day, the probability should very likely increase for zero-valued (highest) preferences and decrease for three-valued preferences towards midday. The probability distributions which are used here— depending on the start time of the course—are given in Table 3. Similarly, the number of different non-zero weights λ_i^* as well as the weights themselves are drawn—where needed—at random from a uniform distribution.

Using such preference data, the first out of the three questions mentioned above to be addressed in detail here is whether the CPU times to solve the extended model are short enough to make the extended problem amenable to an automated solution,

Table 4 Default parameter setting

Parameter	Symbol	Value
Fraction of students deviating from default preferences		30%
Set of lunch break periods	L_{id}	{12, 13, 14}
Morning core time end	e_{id}^M	14
Afternoon core time start	f_{id}^A	13
Maximum presence days	m_i	4
Individual welfare upper bound	$\overline{z}_i$	0.5
Penalty for individual welfare upper bound	α_i	100
Penalty for unbalanced parallel group utilisation	β_j	1
Minimal-group-capacity-utilisation factor	κ_{jk}	0.75
Best magnitude of group preferences	$\underline{\Pi}_i$	0
Worst magnitude of group preferences	$\overline{\Pi}_i$	$3 \cdot \lvert J_i \rvert$
Best magnitude for lunch break violations	$\underline{l}_i$	0
Worst magnitude for lunch break violations	$\overline{l}_i$	5
Best magnitude of maximum presence days violation	$\underline{r}_i$	0
Worst magnitude of maximum presence days violation	$\overline{r}_i$	1
Best magnitude of assignment outside core times	$\underline{w}_i^M$, $\underline{w}_i^A$	0
Worst magnitude of assignment outside core times	$\overline{w}_i^M$, $\overline{w}_i^A$	75

too. In particular, it could be conjectured that the CPU times depend strongly on the fraction of students with preferences deviating from the default weights (100 points on group preferences). For this purpose and for the following sensitivity tests, a reference case is needed which is defined by the default parameter settings given in Table 4.

Then the percentage of students without default weighting can be varied parametrically in steps of 10 % while all other parameters stay at their default value, i.e. the sensitivity analyses are carried out ceteris paribus—changing one parameter at a time only.

As the second question, it is examined what kind of effects different settings of the individual-contribution upper bounds $\overline{z}_i$ have on the total of the individual objective contributions and on the number of upper bound violations. These effects are analysed for jointly varied upper bounds $\overline{z}_i$ in the range from 0 to 1 incremented by 0.1 where it should be noted that the individual objectives are normalised to be between 0 and 1.

The third aspect to be examined here is the influence of the minimal-group-capacity-utilisation factors κ_{jk} on again the total of the individual objective contributions and on the mean variance of the resulting group utilizations. This parameter is also varied in steps of 0.1 between 0 and 1.

5.3.2 Results

Altogether, there are 32 experiments (including the case of default parameter settings) which need to be performed to address these three questions. For each of these experi-

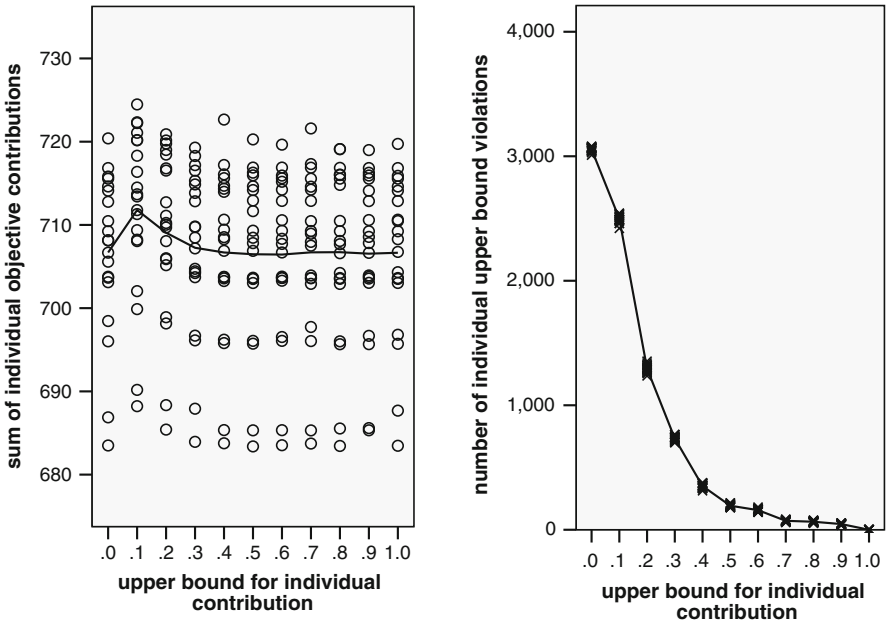

Fig. 3 Effects of different individual objective contribution upper bounds $\overline{z}_i$

ments with a different parameter setting, 20 runs with independent preference realisations (in total 640 optimisation runs) are evaluated to adequately take into account any fluctuations of the artificially generated preferences. Mean values and interpolations are calculated over the results of these runs.

The CPU runtimes vary between 78 s and 285 s where all GAMS parameters (except optimality tolerance which is 1 %) are set to their default values. Hence, the student-scheduling problem with the model extensions can easily be solved in operable time for the case of the Business programme in Hamburg. While earlier experiments seemed indeed to support the conjecture that the CPU times depend on the fraction of students with preferences deviating from the default weights for the schedule of a different semester (Heitmann and Brüggemann 2012), extensive tests of this hypothesis cannot establish such a relation significantly for the timetabling and registration data of the summer semester 2011.

Next, the question is investigated how different individual objective value upper bounds $\overline{z}_i$ effect the total of individual objective contribution and the number of individual upper bound violations. This is illustrated in Fig. 3 where the left panel shows, for each of the 20 runs in each experiment, the sum of all individual objective contributions (circles). Corresponding averages over the same 20 runs are also displayed and given as an interpolation line. The total preference satisfaction is not much affected by changes in the upper bounds on the individual objective contributions. It should be pointed out that this sum of all individual objective contributions is not the objective function which additionally includes compensations for

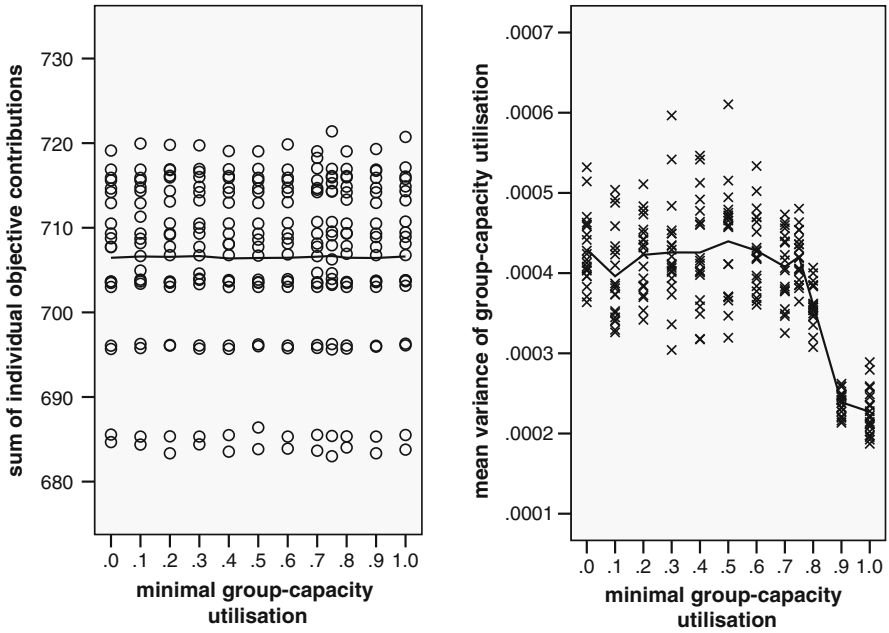

Fig. 4 Effects of different minimal-group-capacity-utilisation factors κ_{jk}

violations of the individual-upper-bound and minimal-group-capacity-utilisation soft constraints. In contrast, the number of individual upper bound violations (crosses in the right panel) increases rapidly as these bounds become tighter, i.e. smaller. This growth in the sheer number of violations, however, appears to be outbalanced by the smaller and smaller individual contributions which explains why the overall effect of these upper bounds on the total of the individual contributions is as observed negligible. For the default parameter setting of the individual upper bounds $\overline{z}_i = 0.5$, the number of violations is only about 5 % of all possible and the individual preferences are fulfilled entirely for all other students in this case.

Thirdly, the effects of the minimal-group-capacity-utilisation factor κ_{jk} are analysed. In the left panel of Fig. 4, the sum of all individual objective contributions (circles) is plotted against different values of the minimal-group-capacity-utilisation factor again for 20 runs in each experiment where the mean values over these 20 runs are shown by an interpolation line, too. As in the previous case, the total satisfaction exhibits a nearly constant behaviour while the mean variance of the actual group-capacity utilisations (crosses in the right panel) is roughly unchanged only for smaller values of this factor before a threshold of approximately 75 % (this is also the default value for κ_{jk}) is reached and declines rapidly thereafter. This reduced mean variance corresponds to a more homogenous utilisation of multiple groups and it is remarkable that it is possible—for the timetabling and data setting under scrutiny—to improve this actual group-capacity utilisation without noticeable effects in total students' satisfaction.

6 Conclusion and further work

In this paper, a successful approach to the student scheduling problem is presented. The operational process includes a feasibility test before assigning the students. Employees in university administration can be informed about infeasible enrolments before the students are assigned and can try to resolve such issues early on. Feasible schedules can then be guaranteed—in contrast to the status quo with the currently used campus-management software alone. Besides this guarantee that the resulting timetables are clash free on an individual basis, students can specify their individual preferences in a model extension by choosing between individual goals for the scheduling, like group preferences, lunch-time breaks, core times, and maximum number of presence days at the university.

Although the approach is very promising, it is currently not yet being used in practice. This is mainly due to staff shortages which have hindered setting up appropriate interfaces in the campus management software for the data ex- and import. As a consequence, real preference values have not yet been specified by the students. An experimental study is carried out in two stages for both model versions: first, the basic model is tested using real-life timetabling and registration data from the Business programme of Universität Hamburg for the student scheduling problem. Secondly, the same framework is used along with artificially generated preferences to analyse the extended model. The problems of either stage are easily solvable in operable time for these instances.

As a result of this approach, the sub-cohorts (i. e., the division of students and multiple teaching groups) used in the Department of Business Administration would become obsolete, and the student scheduling could be improved. University administration effort could be reduced as well if the sub-cohorts were abandoned. As a further consequence, the course timetabling could be automated similarly to the timetabling approach by Schimmelpfeng and Helber (2007) in an additional step.

Further research should be directed to analyse the sensitivity of the suggested solution approach to other combinations of parameter settings for the penalty costs, and dependencies between individual welfare upper bounds and the group utilisation factors. Moreover, it would be interesting to perform scheduling tests for larger units—up to the entire Universität Hamburg with roughly 40,000 students—to find out what kind of instance size can still be solved exactly within a satisfactory time frame despite the problem being NP-hard.

Acknowledgments This work was partially funded by the Fachbereich Betriebswirtschaftslehre (Department of Business Administration) of the Universität Hamburg from student tuition fees. This is gratefully acknowledged as well as the advice and support provided by the Fachbereich and the company Datenlotsen in Hamburg. In addition, the authors wish to thank the Associate Editor and the three anonymous referees for their valuable comments and suggestions in two revisions which greatly improved the paper.

Appendix

Table 5 List of notation

Symbol	Description
α_i	Weight in the objective of individual upper bound violation for student i
β_j	Weight in the objective of unbalanced group utilisation for course j
b_{ihd}	Indicator of blocked time slot for student i at time h on day d
e_{id}^M	End of morning core time for student i on day d
f_{id}^A	Sart of afternoon core time for student i on day d
d	Day
D	Set of days
c_{ihd}	Indicator if there is a possibility of a clash for student i at time slot h on day d
Π_i	Total group preference for student i
$\underline{\Pi}_i$, $\overline{\Pi}_i$	Best and worst possible values of total group preference of student i
h	Hour (index of time slot)
H	Set of hours
H_{id}^A	Set of afternoon core time slots of student i on day d
H_{id}^M	Set of morning core time slots of student i on day d
i	Student
I	Set of students
I_j	Set of students who take course j
j	Course
J	Set of courses
J_i	Set of courses taken by student i
κ_{jk}	Minimal capacity utilisation of group k in course j
k, $\hat{k}$	Group
K	Set of groups
K_j	Set of groups of course j
λ_i^l	Weight of normalised total lunch-break violations for student i in the individual contribution to the objective
λ_i^Π	Weight of normalised total group preferences for student i in the individual contribution to the objective
λ_i^r	Weight of normalised violation of maximum-number-of-days-per-week-at-the-university constraint for student i in the individual contribution to the objective
$\lambda_i^{w^A}$	Weight of normalised violation of assignments-before-afternoon-core-time constraint for student i in the individual contribution to the objective
$\lambda_i^{w^M}$	Weight of normalised violation of assignments-after-morning-core-time constraint for student i in the individual contribution to the objective
l_{id}	Variable indicating a lunch break violation for student i on day d
$\underline{l}_i$, $\overline{l}_i$	Best and worst number of lunch-break violations for student i
L_{id}	Set of lunch break time slots of student i on day d
m_i	Preferred maximum number of days at the university for student i

Table 5 continued

Symbol	Description
π_{ijk}	Group preference of student i for group k in course j
p_{ij}	Variable indicating infeasible enrolment of student i for course j
q_{jk}	Group capacity for group k of course j
r_i	Variable indicating violation of maximum number of days per week at university for student i
$\underline{r}_i$, $\overline{r}_i$	Best and worst number of violations of maximum number of days constraint for student i
s_{jk}	Variable indicating unbalanced group utilisation of group k in course j
t_{jkhd}	Indicator that group k of course j is taught at time slot h on day d
u_i	Variable individual upper bound violation for student i
w_i^A	Variable of time-weighted assignments before afternoon core times violations for student i
$\underline{w}_i^A$, $\overline{w}_i^A$	Best and worst values for variable w_i^A
w_i^M	Variable of time-weighted assignments after morning core times violations for student i
$\underline{w}_i^M$, $\overline{w}_i^M$	Best and worst values for variable w_i^M
x_{ijk}	Binary variable indicating whether student i is assigned to group k of course j
y_{id}	Binary variable indicating whether student i is assigned to any course on day d
z_i	Variable for individual contribution of student i to the objective
$\overline{z}_i$	Upper bound for the individual contribution of student i to the objective

References

Burke E, Jackson K, Kingston JH, Weare R (1997) Automated university timetabling: the state of the art. Comput J 40(9):565–571

Carter MW, Laporte G (1996) Recent developments in practical examination timetabling. In: Practice and theory of automated timetabling. Lecture notes in computer science, vol 1153. Springer, Berlin, pp 1–21

Cheng E, Kruk S, Lipman M (2003) Flow formulations for the student scheduling problem. In: Burke E, De Causmaecker P (eds) Practice and theory of automated timetabling IV. Lecture notes in computer science, vol 2740. Springer, Berlin, pp 299–309

Daskalaki S, Birbas T (2005) Efficient solutions for a university timetabling problem through integer programming. Eur J Oper Res 160(1):106–120 (applications of Mathematical Programming Models)

Datenlotsen (2013) CampusNet. http://www.datenlotsen.de/index.php/de/produkte/campusnet (last download on 25 April 2013 at 1:30 pm)

de Werra D (1985) An introduction to timetabling. Eur J Oper Res 19(2):151–162

Drexl A, Salewski F (1997) Distribution requirements and compactness constraints in school timetabling. Eur J Oper Res 102(1):193–214

Feldman R, Golumbic MC (1989) Constraint satisfiability algorithms for interactive student scheduling. In: Proceedings of the eleventh international joint conference on artificial intelligence. Springer, Berlin, pp 1010–1016

French S (1989) Decision theory: an introduction to the mathematics of rationality. In: Mathematics and its applications (Ellis Horwood Ltd). Prentice Hall, Englewood Cliffs

Garey MR, Johnson DS (1979) Computers and intractability: a guide to the theory of NP-completeness. W. H. Freeman and Company, New York (addendum updated March 1991)

Heitmann H, Brüggemann W (2012) Experimental study of the students-scheduling problem at the Department of Business Administration, Universität Hamburg, winter semester 2010/11. http://www.uni-hamburg.de/or/pbas (last visit: 06 May 2013, 8.00 p.m)

Laporte G, Desroches S (1986) The problem of assigning students to course sections in a large engineering school. Comput Oper Res 13(4):387–394

Lewis R, Paechter B (2007) Finding feasible timetables using group-based operators. IEEE Trans Evol Comput 11(3):397–413

Schaerf A (1999) A survey of automated timetabling. Artif Intell Rev 13(2):87–127

Schimmelpfeng K, Helber S (2007) Application of a real-world university-course timetabling model solved by integer programming. OR Spectr 29:783–803

Universität Hamburg, Administration (2011) 2nd and 4th semester courses and groups of the summer semester 2011. http://www.wiso.uni-hamburg.de/fileadmin/einrichtungen/studienbuero_wiwi/Vorlesungsverzeichnis/2011-Sommersemester/GESAMT-UEBERSICHT_Bachelor-Uebungsgruppen_SoSe2011.pdf (last visit: 19 Nov 2012, 5.00 p.m)

van den Broek J, Hurkens C, Woeginger G (2009) Timetabling problems at the TU Eindhoven. Eur J Oper Res 196(3):877–885

Young HP (1995) Equity: in theory and practice. Princeton University Press, Princeton